Thread-Safe Singleton Pattern

Theory, Edge Cases & Code Examples

- ✓ Singleton Pattern Overview
- ✓ C++ Singleton Implementation Evolution
- ✓ Thread Safety Challenges
- ✓ Implementation Patterns Comparison
- ✓ Autonomous Vehicle Example

→ *Swipe to learn* →

Thread-Safe Singleton Pattern

Theory

📌 1. Singleton Pattern Overview

Definition: Creational design pattern ensuring exactly one instance of a class with global access point.

Core Guarantee:

Requirement	Implementation	Purpose
Single instance	Private constructor	Prevent external instantiation
Global access	Static getInstance()	Provide controlled access point
Lazy initialization	Create on first use	Defer resource allocation
Thread safety	C++11 static guards or mutex	Prevent race conditions

When to Use Singleton:

Common Use Cases:

```
class ConfigManager {           // ✅ One configuration per application
class DatabasePool {             // ✅ Single connection pool
class Logger {                   // ✅ Centralized logging
class HardwareInterface {        // ✅ One driver per physical device
class ServiceRegistry {          // ✅ Global service locator
```

Inappropriate Uses:

```
class User {                     // ❌ Multiple users exist
class Transaction {              // ❌ Many transactions concurrently
class Sensor {                   // ❌ May need multiple sensors
```

Thread-Safe Singleton Pattern

Theory

📌 2. C++ Singleton Implementation Evolution (1/2)

Historical Progression:

Era	Approach	Thread Safety	Issue
Pre-C++11	Static member + manual init	❌ Racy	Requires double-checked locking (broken)
Pre-C++11	Meyers Singleton	❌ Not guaranteed	Compiler-dependent behavior
C++11+	Meyers Singleton	✅ Guaranteed	Static local thread-safe (§6.7/4)
C++11+	std::call_once	✅ Guaranteed	Explicit one-time initialization

Modern Best Practice (Meyers Singleton):

```
class Singleton {
private:
    Singleton() = default;                          // Private constructor
    Singleton(const Singleton&) = delete;           // Delete copy
    Singleton& operator=(const Singleton&) = delete; // Delete assign

public:
    static Singleton& getInstance() {
        static Singleton instance;  // ✅ Thread-safe in C++11+
        return instance;            // ✅ Lazy initialization
    }                               // ✅ Auto cleanup at exit
};
```

Why This Works (C++11 Guarantee):

C++ Standard §6.7/4: "If control enters the declaration concurrently while the variable is being initialized, the concurrent execution shall wait for completion of the initialization."

Thread-Safe Singleton Pattern

Theory

📌 2. C++ Singleton Implementation Evolution (2/2)

Compiler Implementation (conceptual):

```
// What the compiler generates internally:
static bool initialized = false;
static mutex init_mutex;
static aligned_storage<Singleton> storage;

if (!initialized) {
    lock_guard<mutex> lock(init_mutex);
    if (!initialized) {
        new (&storage) Singleton();  // Construct in-place
        initialized = true;
    }
}
return reinterpret_cast<Singleton&>(storage);
```

Thread-Safe Singleton Pattern

Theory

📌 3. Thread Safety Challenges (1/2)

Three Critical Concerns:

A. Initialization Race:

Problem	Solution	C++ Version
Multiple threads create multiple instances	Meyers Singleton (static local)	C++11+
	std::call_once	C++11+
	Mutex-protected check	All versions

B. Static Initialization Order Fiasco:

```
// File1.cpp
Logger logger;  // Global static - when initialized?

// File2.cpp
Config config;  // Constructor uses logger - UNDEFINED ORDER!

// ✅ SOLUTION: Use Meyers Singleton
Logger& getLogger() {
    static Logger instance;  // Initialized on first use
    return instance;
}
```

Thread-Safe Singleton Pattern

Theory

📌 3. Thread Safety Challenges (2/2)

C. Destruction Order Problem:

```
// Service destroyed AFTER Logger?
~Service() {
    Logger::getInstance().log("Destroying");  // ❌ Use-after-destruction!
}

// ✅ SOLUTION 1: Phoenix Singleton (never destroy)
Logger& getInstance() {
    static Logger* instance = new Logger();  // Intentional leak
    return *instance;
}

// ✅ SOLUTION 2: Explicit shutdown phase
class App {
    void shutdown() {
        service.reset();   // Destroy in controlled order
        logger.reset();
    }
};
```

Thread-Safe Singleton Pattern

Theory

📌 4. Implementation Patterns Comparison

Pattern	Code Skeleton	Pros	Cons
Meyers	`static T instance;`	Simple, fast, thread-safe (C++11)	Hard to test, destruction order issues
std::call_once	`call_once(flag, init)`	Explicit control, exception-safe	More verbose, slight overhead
Mutex + heap	`lock + unique_ptr`	Testable (can reset), flexible	Lock on every access, slower
Atomic CAS	`atomic<T*> + compare_exchange`	Lock-free after init	Complex, error-prone, rarely needed
Phoenix	`static T* = new T()`	Safe during destruction	Memory leak (intentional)

Thread-Safe Singleton Pattern

Theory

📌 5. Autonomous Vehicle Example

Real-World Singleton Use Cases:

```
// ✅ Sensor Manager - One instance manages all sensors
class SensorManager {
    std::vector<Sensor*> sensors;
    mutable std::mutex mtx;

    SensorManager() { /* Initialize CAN bus */ }

public:
    static SensorManager& getInstance() {
        static SensorManager instance;
        return instance;
    }

    void registerSensor(Sensor* s) {
        std::lock_guard lock(mtx);
        sensors.push_back(s);
    }
};

// ✅ CAN Bus Driver - Single physical interface
class CANBusDriver {
    int bus_fd;  // File descriptor to /dev/can0

    CANBusDriver() {
        bus_fd = open("/dev/can0", O_RDWR);
        if (bus_fd < 0) throw std::runtime_error("CAN init failed");
    }

public:
    static CANBusDriver& getInstance() {
        static CANBusDriver instance;  // ✅ Retries if throws
        return instance;
    }

    void sendFrame(const CANFrame& frame) {
        write(bus_fd, &frame, sizeof(frame));
    }
};
```

Thread-Safe Singleton Pattern

Theory

📌 6. Why Singleton Matters

Critical Concepts Demonstrated:

Concept	How Singleton Tests It	Interview Relevance
Static initialization	C++11 §6.7/4 guarantees	Understanding language spec
Thread safety	Race conditions, memory barriers	Concurrency fundamentals
Memory management	Heap vs stack, RAII, leaks	Resource ownership
Destruction order	Static lifetime, fiasco patterns	Undefined behavior awareness
Design tradeoffs	Testability vs simplicity	Software engineering judgment

Common Interview Questions:

- "Is double-checked locking safe in C++11?" (No, use Meyers)
- "How do you test Singletons?" (Dependency injection, reset hooks)
- "What's the static initialization order fiasco?" (Undefined order across TUs)
- "Can Singleton be thread-safe without mutexes?" (Yes, Meyers in C++11)

Thread-Safe Singleton Pattern

Edge Cases

📌 1. Double-Checked Locking Anti-Pattern (Pre-C++11)

```
// ❌ BROKEN: Classic double-checked locking (pre-C++11)
class Singleton {
    static Singleton* instance;
    static std::mutex mtx;

    Singleton() {}

public:
    static Singleton* getInstance() {
        if (!instance) {  // First check (unprotected)
            std::lock_guard<std::mutex> lock(mtx);
            if (!instance) {  // Second check (protected)
                instance = new Singleton();  // ❌ Not atomic!
            }
        }
        return instance;
    }
};
```

This pattern is broken because the compiler can reorder instructions, potentially returning a partially constructed object. The assignment `instance = new Singleton()` involves three steps: allocate memory, construct object, assign pointer. Without memory barriers, another thread might see a non-null pointer to an unconstructed object.

Thread-Safe Singleton Pattern

Edge Cases

📌 2. Static Initialization Order Fiasco

```
// ❌ DANGEROUS: Static member initialization
class Logger {
    static Logger instance;  // When is this initialized?
    Logger() { /* ... */ }
};

class Config {
    Config() {
        Logger::instance.log("Config created");  // ❌ Undefined behavior if Logger not initialized yet
    }
};

Logger Logger::instance;  // Order relative to Config construction is undefined
```

Static members across translation units have undefined initialization order, leading to use-before-construction bugs that are nearly impossible to debug.

Thread-Safe Singleton Pattern

Edge Cases

📌 3. Meyer's Singleton and Function-Static Destruction

```
// ✅ Thread-safe but watch destruction order
class DatabaseConnection {
public:
    static DatabaseConnection& getInstance() {
        static DatabaseConnection instance;  // ✅ C++11 guarantees thread-safe initialization
        return instance;
    }

    ~DatabaseConnection() {
        // ❌ DANGER: What if another static destructor calls getInstance()?
        cleanup();
    }
};
```

While initialization is thread-safe, destruction happens in reverse order of construction. If a destructor of another static object tries to access this Singleton after it's destroyed, you get undefined behavior.

Thread-Safe Singleton Pattern

Edge Cases

📌 4. Singleton with Lazy Initialization and shared_ptr

```
// ✅ Thread-safe with std::call_once
class ResourceManager {
    static std::shared_ptr<ResourceManager> instance;
    static std::once_flag initFlag;

    ResourceManager() = default;

public:
    static std::shared_ptr<ResourceManager> getInstance() {
        std::call_once(initFlag, []() {
            instance = std::shared_ptr<ResourceManager>(new ResourceManager());
        });
        return instance;
    }
};

std::shared_ptr<ResourceManager> ResourceManager::instance = nullptr;
std::once_flag ResourceManager::initFlag;
```

Using `shared_ptr` allows controlled lifetime but adds atomic reference counting overhead. The `std::call_once` ensures thread-safe initialization without double-checked locking pitfalls.

Thread-Safe Singleton Pattern

Edge Cases

📌 5. Singleton Reset/Mock for Testing

```
// Testing challenge: How to reset Singleton between tests?
class ServiceLocator {
    static std::unique_ptr<ServiceLocator> instance;

public:
    static ServiceLocator& getInstance() {
        if (!instance) instance = std::make_unique<ServiceLocator>();
        return *instance;
    }

    // ⚠️ For testing only - not thread-safe
    static void reset() {
        instance.reset();
    }
};
```

Testing Singletons is notoriously difficult. Adding reset functionality breaks the Singleton guarantee and isn't thread-safe. Better approaches use dependency injection or test-specific factory methods.

Thread-Safe Singleton Pattern

Code Examples

📌 1. Easy - Classic Non-Thread-Safe Singleton

This Meyers Singleton is thread-safe in C++11+ due to guaranteed thread-safe static local initialization. The function-local static is created on first call and destroyed at program exit.

```
#include <iostream>

class ConfigManager {
private:
    int configValue;

    ConfigManager() : configValue(42) {  // Private constructor
        std::cout << "ConfigManager created\n";
    }

    // ❌ Delete copy constructor and assignment
    ConfigManager(const ConfigManager&) = delete;
    ConfigManager& operator=(const ConfigManager&) = delete;

public:
    static ConfigManager& getInstance() {
        static ConfigManager instance;  // ✅ Created on first call
        return instance;
    }

    int getValue() const { return configValue; }
    void setValue(int val) { configValue = val; }
};

int main() {
    ConfigManager& cfg1 = ConfigManager::getInstance();
    ConfigManager& cfg2 = ConfigManager::getInstance();

    std::cout << "Same instance? " << (&cfg1 == &cfg2) << "\n";  // Prints: 1

    cfg1.setValue(100);
    std::cout << "cfg2 value: " << cfg2.getValue() << "\n";  // Prints: 100
}
```

Thread-Safe Singleton Pattern

Code Examples

📌 2. Mid - Thread-Safe Singleton with Mutex Protection (1/2)

This version uses heap allocation with `unique_ptr` and protects both getInstance() and log() with mutex. More flexible than Meyers Singleton but adds locking overhead.

```
#include <iostream>
#include <mutex>
#include <memory>

class Logger {
private:
    static std::unique_ptr<Logger> instance;
    static std::mutex mtx;
    int logCount;

    Logger() : logCount(0) {
        std::cout << "Logger initialized\n";
    }

public:
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;

    static Logger& getInstance() {
        std::lock_guard<std::mutex> lock(mtx);  // ✅ Fully protected
        if (!instance) {
            instance.reset(new Logger());
        }
        return *instance;
    }

    void log(const std::string& msg) {
        std::lock_guard<std::mutex> lock(mtx);
        std::cout << "[Log " << ++logCount << "] " << msg << "\n";
    }
};

std::unique_ptr<Logger> Logger::instance = nullptr;
std::mutex Logger::mtx;

int main() {
// ...continues on next slide
```

Thread-Safe Singleton Pattern

Code Examples

📌 2. Mid - Thread-Safe Singleton with Mutex Protection (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    Logger& log1 = Logger::getInstance();
    Logger& log2 = Logger::getInstance();

    log1.log("System started");
    log2.log("Configuration loaded");  // Same instance
}
```

Thread-Safe Singleton Pattern

Code Examples

📌 3. Mid - std::call_once Initialization (1/2)

`std::call_once` provides clean, exception-safe initialization without manual mutex management. The lambda runs exactly once even if multiple threads call getInstance() simultaneously.

```
#include <iostream>
#include <mutex>
#include <memory>

class DatabaseConnection {
private:
    static std::unique_ptr<DatabaseConnection> instance;
    static std::once_flag initFlag;

    std::string connectionString;

    DatabaseConnection(const std::string& connStr) : connectionString(connStr) {
        std::cout << "DB connected: " << connStr << "\n";
    }

public:
    DatabaseConnection(const DatabaseConnection&) = delete;
    DatabaseConnection& operator=(const DatabaseConnection&) = delete;

    static DatabaseConnection& getInstance() {
        std::call_once(initFlag, []() {
            // ✅ Guaranteed to run exactly once, thread-safe
            instance.reset(new DatabaseConnection("localhost:5432"));
        });
        return *instance;
    }

    void query(const std::string& sql) {
        std::cout << "Executing: " << sql << "\n";
    }
};

std::unique_ptr<DatabaseConnection> DatabaseConnection::instance = nullptr;
std::once_flag DatabaseConnection::initFlag;

int main() {
// ...continues on next slide
```

Thread-Safe Singleton Pattern

Code Examples

📌 3. Mid - std::call_once Initialization (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    auto& db = DatabaseConnection::getInstance();
    db.query("SELECT * FROM users");
}
```

Thread-Safe Singleton Pattern

Code Examples

📌 4. Advanced - Lazy Initialization with std::shared_ptr (1/2)

Using `shared_ptr` allows multiple references and controlled destruction. The atomic counters avoid mutex overhead for statistics tracking. Double-checked locking works correctly with `shared_ptr` due to its atomic operations.

```
#include <iostream>
#include <memory>
#include <mutex>
#include <atomic>

class CacheManager {
private:
    static std::shared_ptr<CacheManager> instance;
    static std::mutex mtx;
    std::atomic<int> hits{0};
    std::atomic<int> misses{0};

    CacheManager() {
        std::cout << "CacheManager initialized\n";
    }

public:
    CacheManager(const CacheManager&) = delete;
    CacheManager& operator=(const CacheManager&) = delete;

    static std::shared_ptr<CacheManager> getInstance() {
        // ✅ Double-checked locking CORRECT way with shared_ptr
        if (!instance) {
            std::lock_guard<std::mutex> lock(mtx);
            if (!instance) {
                instance = std::shared_ptr<CacheManager>(new CacheManager());
            }
        }
        return instance;
    }

    void recordHit() { ++hits; }
    void recordMiss() { ++misses; }

    void printStats() const {
        std::cout << "Hits: " << hits << ", Misses: " << misses << "\n";
// ...continues on next slide
```

Thread-Safe Singleton Pattern

Code Examples

📌 4. Advanced - Lazy Initialization with std::shared_ptr (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
};

std::shared_ptr<CacheManager> CacheManager::instance = nullptr;
std::mutex CacheManager::mtx;

int main() {
    auto cache1 = CacheManager::getInstance();
    auto cache2 = CacheManager::getInstance();

    cache1->recordHit();
    cache2->recordMiss();

    cache1->printStats();  // Prints: Hits: 1, Misses: 1
}
```

Thread-Safe Singleton Pattern

Code Examples

📌 5. Advanced - Meyer's Singleton with CRTP for Multiple Singletons (1/2)

CRTP (Curiously Recurring Template Pattern) creates a reusable Singleton base class. Each derived class gets its own thread-safe static instance without code duplication.

```
#include <iostream>
#include <string>

template <typename T>
class Singleton {
protected:
    Singleton() = default;

public:
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;

    static T& getInstance() {
        static T instance;  // ✅ Thread-safe in C++11+
        return instance;
    }
};

class AudioManager : public Singleton<AudioManager> {
    friend class Singleton<AudioManager>;  // Allow base to construct

private:
    int volume = 50;
    AudioManager() { std::cout << "AudioManager created\n"; }

public:
    void setVolume(int vol) { volume = vol; }
    int getVolume() const { return volume; }
};

class NetworkManager : public Singleton<NetworkManager> {
    friend class Singleton<NetworkManager>;

private:
    std::string ipAddress = "127.0.0.1";
    NetworkManager() { std::cout << "NetworkManager created\n"; }
// ...continues on next slide
```

Thread-Safe Singleton Pattern

Code Examples

📌 5. Advanced - Meyer's Singleton with CRTP for Multiple Singletons (2/2)

[Code continues across slides]

```
// ...continued from previous slide

public:
    void setIP(const std::string& ip) { ipAddress = ip; }
    std::string getIP() const { return ipAddress; }
};

int main() {
    auto& audio = AudioManager::getInstance();
    audio.setVolume(75);

    auto& network = NetworkManager::getInstance();
    network.setIP("192.168.1.100");

    std::cout << "Audio volume: " << AudioManager::getInstance().getVolume() << "\n";
    std::cout << "Network IP: " << NetworkManager::getInstance().getIP() << "\n";
}
```

Thread-Safe Singleton Pattern

Code Examples

📌 6. Advanced - Singleton with Dependency Injection for Testing (1/2)

This design maintains Singleton benefits while enabling testability through dependency injection. The abstract interface allows mock implementations during testing.

```
#include <iostream>
#include <memory>

class ILogger {
public:
    virtual ~ILogger() = default;
    virtual void log(const std::string& msg) = 0;
};

class ProductionLogger : public ILogger {
public:
    void log(const std::string& msg) override {
        std::cout << "[PROD] " << msg << "\n";
    }
};

class TestLogger : public ILogger {
public:
    void log(const std::string& msg) override {
        std::cout << "[TEST] " << msg << "\n";
    }
};

class LoggerFactory {
private:
    static std::unique_ptr<ILogger> instance;

public:
    static ILogger& getInstance() {
        if (!instance) {
            instance = std::make_unique<ProductionLogger>();
        }
        return *instance;
    }

    // ✅ For testing: inject mock implementation
// ...continues on next slide
```

Thread-Safe Singleton Pattern

Code Examples

📌 6. Advanced - Singleton with Dependency Injection for Testing (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    static void setInstance(std::unique_ptr<ILogger> logger) {
        instance = std::move(logger);
    }

    static void reset() {
        instance.reset();
    }
};

std::unique_ptr<ILogger> LoggerFactory::instance = nullptr;

int main() {
    // Production usage
    LoggerFactory::getInstance().log("Application started");

    // Test usage
    LoggerFactory::setInstance(std::make_unique<TestLogger>());
    LoggerFactory::getInstance().log("Running unit test");

    LoggerFactory::reset();
}
```

Thread-Safe Singleton Pattern

Code Examples

📌 8. Performance - Meyer's Singleton Benchmark (1/2)

Meyers Singleton has minimal performance overhead after first initialization. The static local variable is initialized with thread-safe guards, but subsequent accesses are lock-free.

```
#include <iostream>
#include <chrono>
#include <thread>
#include <vector>

class FastSingleton {
public:
    static FastSingleton& getInstance() {
        static FastSingleton instance;  // ✅ Initialized once, no locking after first call
        return instance;
    }

    void doWork() { /* fast operation */ }

private:
    FastSingleton() = default;
};

void workerThread(int iterations) {
    for (int i = 0; i < iterations; ++i) {
        FastSingleton::getInstance().doWork();
    }
}

int main() {
    const int iterations = 1000000;
    const int numThreads = 4;

    auto start = std::chrono::high_resolution_clock::now();

    std::vector<std::thread> threads;
    for (int i = 0; i < numThreads; ++i) {
        threads.emplace_back(workerThread, iterations);
    }

    for (auto& t : threads) {
// ...continues on next slide
```

Thread-Safe Singleton Pattern

Code Examples

📌 8. Performance - Meyer's Singleton Benchmark (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        t.join();
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::cout << "Completed " << (iterations * numThreads) << " calls in "
              << duration.count() << "ms\n";
}
```

What You Learned

Key Takeaways

- ✓ Singleton Pattern Overview
- ✓ C++ Singleton Implementation Evolution
- ✓ Thread Safety Challenges
- ✓ Implementation Patterns Comparison

→ Swipe to learn →